Pre-Class Knowledge Base

Multi-Agent, Conversational, and Multimodal Agentic AI

Ready-reckoner preparation material for the Week 5 live class and hands-on project.

Multi-Agent, Conversational, and Multimodal Agentic AI

Purpose: A ready reckoner for the prerequisite topics shown in the pre-class slide. Read the concepts first, then complete the hands-on checkpoints before the live class.

Course context: This guide prepares you for the Week 5 multimodal conversational assistant project. The reference project, AxiomCart, combines LangGraph orchestration, product RAG, support tools, parallel specialist agents, conversation persistence, human-in-the-loop interaction, and voice I/O.

---

1. Learning Map

The topics are connected as a single engineering stack:

1. **LLMs** interpret requests and generate responses.
2. **Tools and tool calls** let an LLM take controlled actions.
3. **LangChain** supplies reusable model, prompt, message, tool, and retrieval components.
4. **RAG** grounds answers in external or private knowledge.
5. **Vector databases** make semantic retrieval practical.
6. **LangGraph** turns model calls and tools into explicit stateful workflows.
7. **Parallelisation** lets independent specialists work at the same time.
8. **Multi-agent systems** divide responsibilities between focused agents.
9. **Persistence** carries state across turns and supports resumable workflows.

A useful design rule is: use the simplest architecture that meets the requirement. Multi-agent coordination is valuable when specialization, isolation, parallel work, or workflow control outweighs the extra complexity.

---

2. LLMs

What an LLM does

A large language model predicts the next token from the conversation context. In an application, it is used to classify, extract, plan, call tools, summarize, transform, and generate content. It does not automatically know the current state of your database or the latest facts on the internet.

Message roles

• **System message:** Stable behavior, scope, safety rules, output format, and tool-use policy.

• **Human message:** The user's request or a task delegated by another node.

• **AI message:** The model's response, which may contain text or tool calls.

• **Tool message:** The result returned after a tool call.

A tool-calling loop usually looks like:

```
human request -> model decides -> tool call -> tool result -> model decides again -> final answer
```

Model configuration

Important controls include:

• `model`: the deployed model name.

• `temperature`: lower values are more deterministic; higher values are more varied.

• `max_tokens` or output limits: constrain response length.

• `timeout` and retry policy: make network calls operationally safer.

• Structured output or JSON schema: make routing and extraction easier to validate.

Prompting checklist

A production prompt should state the role, task, available context, allowed tools, constraints, failure behavior, and expected output. Prefer explicit instructions such as “use only the retrieved context” or “return one of these three labels.”

Common failure modes

• Treating generated text as verified fact.

• Sending too much irrelevant conversation history.

• Asking the model to route using labels that are not validated in code.

• Allowing a tool call without input validation or authorization.

• Assuming a model response is always a plain string.

Hands-on checkpoint

Call `ChatOpenAI` with a system message and a human message. Then ask it to classify three requests into `product`, `support`, and `general`. Print the raw response and inspect how message roles affect the result.

---

3. RAG in Agentic Workflows

What RAG is

Retrieval-Augmented Generation adds relevant external context to a model prompt at runtime:

```
source documents -> chunks -> embeddings -> vector search -> retrieved context -> LLM answer
```

RAG is useful when knowledge is private, large, changing, or too specific to rely on model training data.

RAG versus agentic RAG

• **Direct RAG:** Retrieve context for every query using a mostly fixed pipeline.

• **Agentic RAG:** An agent decides whether to retrieve, which retrieval tool to use, whether to refine the query, and whether another action is needed.

Agentic RAG adds flexibility but also adds latency, cost, and more possible failure paths. Tool descriptions and routing prompts must explain when retrieval is necessary.

Retrieval pipeline

1. Load documents and preserve useful metadata.

2. Split documents into meaningful chunks.

3. Embed chunks into vectors.

4. Store vectors and metadata.

5. Embed the user query.

6. Retrieve the most relevant chunks.

7. Place the chunks into a grounded prompt.

8. Generate an answer with citations or source references.

Chunking principles

Chunk by meaning where possible: headings, paragraphs, product records, policies, or FAQ entries. Very small chunks lose context; very large chunks reduce retrieval precision and waste the context window. Store metadata such as source, section, product ID, and date.

Grounding safeguards

• Tell the model to use only supplied context for factual answers.

• Return “I do not have enough information” when retrieval is empty.

• Include source metadata in the response.

• Avoid mixing unrelated retrieved documents.

• Test retrieval separately from generation.

AxiomCart example

The product agent calls `search_product_catalog`. Product data is embedded and searched semantically, then the product agent converts the retrieved products into a useful recommendation. The support agent uses operational tools instead of product RAG for order status.

Hands-on checkpoint

Create five product records, embed them, store them in a local vector database, and retrieve the top three results for a natural-language query such as “wireless headphones under a budget.” Compare semantic search with a keyword-only search and inspect the metadata returned.

---

4. Tools and Tool Calls

What a tool is

A tool is a typed function that an agent can request. The application, not the LLM, executes the function. The model chooses the tool and supplies arguments; the runtime validates and executes it, then sends the result back to the model.

```
from langchain_core.tools import tool

@tool
def get_order_status(order_id: str) -> str:
    """Return the current status for a known order ID."""
    ...
```

Tool design principles

• Give each tool a narrow responsibility.

• Use a precise name and docstring.

• Validate arguments and reject malformed input.

• Return concise, structured results.

• Never expose secrets in tool output.

• Enforce authorization before returning private data.

• Make side-effecting tools explicit and auditable.

Read tools versus write tools

Read tools search or retrieve information. Write tools change data, send messages, create tickets, or place orders. Write tools should usually require confirmation, idempotency controls, and stronger authorization.

Tool-calling loop

A graph node invokes the model. A conditional edge checks whether the latest AI message contains tool calls. If yes, a `ToolNode` executes the requested tools and routes back to the model. If no, the workflow continues to the final answer.

Tool failure handling

Return useful errors such as “order ID not found” rather than raising opaque exceptions. Decide whether the agent should retry, ask the user for clarification, use a fallback, or stop.

Hands-on checkpoint

Implement `get_order_status(order_id)` and `escalate_to_human(reason, priority)`. Test valid input, missing input, an unknown order ID, and a simulated tool failure. Confirm that the model can distinguish a read operation from a side effect.

---

5. LangChain

What LangChain provides

LangChain is a component library and integration layer for LLM applications. Common building blocks include chat models, prompts, messages, output parsers, tools, embeddings, retrievers, document loaders, and runnable pipelines.

Where it fits

LangChain helps build components; LangGraph helps control long-running, stateful workflows. They are complementary rather than competing choices.

Useful abstractions

• `ChatOpenAI`: model client for chat completions.

• Prompt templates: reusable system and user prompt structure.

• Message classes: typed conversation messages.

• `@tool`: expose Python functions to a model.

• Embeddings: convert text into vectors.

• Retrievers: fetch relevant documents.

• `bind_tools`: advertise tools to a model.

• Output parsers or structured output: validate model results.

Runnable composition

LangChain components can be composed into a pipeline where the output of one component becomes the input of another. Keep retrieval, prompt construction, model invocation, and output parsing separable so each stage can be tested independently.

Hands-on checkpoint

Build a small chain that accepts a question, retrieves context, formats a prompt, invokes the model, and returns an answer with a source list. Then convert the retrieval step into a tool that an agent can choose to call.

---

6. LangGraph

Why LangGraph

LangGraph represents an agent workflow as an explicit graph. This is useful when an application needs branching, loops, parallel workers, interrupts, persistence, or inspection of intermediate state.

Core concepts

• **State:** Typed data shared through the workflow.

• **Node:** A Python function that reads state and returns updates.

• **Edge:** A connection that defines the next step.

• **Conditional edge:** A routing function that chooses the next node.

• **Reducer:** A merge rule for updates to a state key.

• **Checkpoint:** A saved snapshot of graph state.

• **Interrupt:** A deliberate pause for human input or approval.

Typical build sequence

1. Define a `TypedDict` state.
2. Decide which fields are replaced and which are accumulated.
3. Implement node functions.
4. Add nodes to a `StateGraph`.
5. Add start, end, normal, and conditional edges.
6. Compile with a checkpointer when persistence is needed.
7. Invoke with a stable `thread_id`.
8. Test each route and failure path.

State design

Keep state explicit and small. Separate user input, route decisions, messages, retrieved context, agent results, approval status, and final output. Use reducers deliberately: message history may append, while a final answer usually replaces the previous value.

AxiomCart topology

```
START -> orchestrator -> product_agent  -+
                         support_agent  -+-> synthesizer -> END
```

Each specialist contains its own model-to-tools loop. The outer graph coordinates the specialists and synthesizes their results.

Hands-on checkpoint

Build a three-node graph: `router`, `specialist`, and `general`. Add a conditional edge from the router, compile the graph, and print the route and final output for three requests.

---

7. Parallelisation

What parallelisation means

Independent work runs concurrently instead of waiting in a serial chain. In an agent workflow, product search and order-status lookup can happen at the same time because neither requires the other's result.

When to parallelise

Use parallel execution when tasks are independent, latency matters, and the downstream node can merge the results. Do not parallelise tasks with strict dependencies, shared mutable state, rate-limit risk, or ordering requirements unless those concerns are handled explicitly.

Fan-out and fan-in

```
                -> worker A -+
router -----                 -> synthesizer
                -> worker B -+
```

The router fans work out. A reducer or result collector merges outputs. The synthesizer performs fan-in and produces one user-facing answer.

LangGraph considerations

• Use state reducers that safely merge concurrent updates.

• Keep worker inputs isolated and serializable.

• Track which worker produced each result.

• Handle partial failure and timeouts.

• Consider rate limits and cost before increasing concurrency.

• Preserve deterministic ordering when presentation order matters.

Hands-on checkpoint

Run two independent async or LangGraph worker tasks: one searches products and one checks an order. Measure serial and parallel elapsed time. Add a timeout and verify that the synthesizer can explain a partial result if one worker fails.

---

8. Multi-Agent Systems

Definition

A multi-agent system coordinates multiple specialized agents or agent-like nodes to solve a task. Each agent can have its own prompt, tools, context, model, and stopping conditions.

Why specialization helps

• A product agent can focus on catalog retrieval and recommendations.

• A support agent can focus on order lookup, policy, and escalation.

• A research agent can focus on web evidence.

• A synthesizer can focus on merging results.

Specialization reduces prompt overload and allows clear ownership of tools and data.

Common patterns

1. **Supervisor/coordinator:** One node delegates work and collects results.
2. **Router:** Classifies a request and sends it to one specialist.
3. **Sequential pipeline:** Each agent transforms the previous result.
4. **Parallel specialists:** Independent agents work concurrently.
5. **Hierarchical system:** A supervisor delegates to subgraphs.
6. **Debate or review:** One agent creates an answer and another critiques it.

Context quarantine

Give agents the smallest context that lets them do their job. The support agent should not receive the entire product catalog, and a product agent should not receive private order records unless needed. This improves focus, privacy, cost, and debuggability.

Coordination costs

Multi-agent systems introduce more model calls, latency, routing errors, state complexity, and observability requirements. Start with one agent and add specialists only when there is a measurable reason.

AxiomCart example

The orchestrator classifies a customer request and may dispatch product and support work in parallel. The product subgraph uses RAG; the support subgraph uses order tools and can interrupt for an order ID; the synthesizer merges one or both results.

Hands-on checkpoint

Design a routing table for five customer questions. For each, specify the responsible agent, allowed tools, required context, expected output, and fallback if the agent cannot answer.

---

9. Vector Databases

What they store

A vector database stores embeddings and associated metadata. Similarity search finds records whose meanings are close to a query, even when exact words differ.

Typical workflow

```
text -> embedding model -> vector + metadata -> vector database
query -> query embedding -> nearest-neighbor search -> documents
```

Important concepts

• **Embedding:** Numeric representation of semantic meaning.

• **Dimension:** Length of the embedding vector.

• **Similarity metric:** Cosine similarity, dot product, or Euclidean distance.

• **Top-k:** Number of results returned.

• **Metadata filter:** Restrict results by fields such as category or tenant.

• **Collection/index:** Logical group of vectors for a use case.

• **Persistence:** Whether the index survives process restart.

Choosing a store

For learning and small local projects, Chroma or FAISS is convenient. For production, consider scale, filtering, durability, access control, backups, latency, and operational support. The database must use embeddings compatible with the index.

Retrieval quality

Measure whether relevant documents appear in the top-k results. Experiment with chunk size, overlap, `k`, metadata filters, similarity search, and MMR. A strong generator cannot repair consistently poor retrieval.

AxiomCart example

The product catalog is embedded and searched through `search_product_catalog`. The retrieved product records are then used by the product agent to create recommendations.

Hands-on checkpoint

Create a local catalog collection, add metadata filters for category and price range, run a semantic query, and inspect the top-k results. Deliberately change the embedding model or collection and observe why compatibility matters.

---

10. Persistence

What persistence solves

Persistence saves conversation messages, routing decisions, tool results, interrupts, and other graph state. It enables follow-up questions, resumable workflows, debugging, and recovery after interruption.

Checkpoints and threads

A checkpointer stores snapshots associated with a `thread_id`. Reusing the same thread ID lets the graph continue a conversation; using a new thread ID starts a separate conversation.

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
graph = workflow.compile(checkpointer=memory)
config = {"configurable": {"thread_id": "customer-123"}}
result = graph.invoke({"user_query": "Where is my order?"}, config=config)
```

`MemorySaver` is useful for development. Production systems generally need a durable checkpointer and clear retention, privacy, and deletion policies.

Conversation memory versus shared store

• **Thread checkpoint:** State for one execution thread or conversation.

• **Shared store:** Data that should be available across threads, such as a user's saved preferences.

Do not put every piece of information into message history. Store durable user facts separately and only retrieve what is relevant.

Human-in-the-loop

A graph can pause with an interrupt when it needs missing information or approval. The application collects the user's answer, resumes the same thread, and continues from the saved state.

Persistence risks

• Storing sensitive data longer than necessary.

• Reusing thread IDs across users.

• Failing to version state schemas.

• Assuming in-memory checkpoints survive a restart.

• Allowing old tool results to override fresh information.

Hands-on checkpoint

Run two turns with the same `thread_id`, ask “What was my last request?”, and verify that the answer uses the saved state. Run the same second query with a new thread ID and confirm that the context is absent. Then simulate an interrupt and resume it.

---

11. End-to-End Preparation Lab

Before the live class, implement this small vertical slice:

1. Create a tiny product catalog and order dataset.

2. Build a vector index for the product catalog.

3. Define `search_product_catalog`, `get_order_status`, and `escalate_to_human` tools.

4. Define a typed LangGraph state with messages and agent results.

5. Add an orchestrator that routes product, support, or combined requests.

6. Run product and support workers in parallel for combined requests.

7. Give each worker a separate model/tool loop.

8. Add a synthesizer for multi-agent responses.

9. Add `MemorySaver` and invoke with a stable thread ID.

10. Add an interrupt when an order ID is missing.

11. Test one product request, one support request, one combined request, a follow-up turn, and one worker failure.

Suggested test inputs

• “Do you have wireless headphones under 5,000?”

• “Where is order ORD102?”

• “My order is delayed and I also want alternatives to the headphones I bought.”

• “I need help with my order.” Then provide an order ID after the graph asks.

• “What did I ask you earlier?”

Definition of done

The system should route correctly, retrieve grounded product data, keep private support context isolated, execute independent work concurrently, synthesize partial or complete results, resume follow-ups using persistence, and expose useful logs when a node or tool fails.

---

12. Quick Reference Table

Topic	Core question	Main implementation object	Typical failure
LLMs	How does the model interpret and generate?	Chat model and messages	Hallucination or uncontrolled output
RAG	How does the answer use external knowledge?	Retriever and grounded prompt	Irrelevant or missing context
Tools	What can the agent do?	Typed Python function	Unsafe or invalid tool call
LangChain	Which reusable components assemble the app?	Models, prompts, tools, retrievers	Tight coupling between stages
LangGraph	How is control flow represented?	StateGraph	Implicit state or routing bugs
Parallelisation	Which tasks can run independently?	Fan-out/fan-in graph	Race conditions or partial failure
Multi-agent	Which specialist owns the task?	Coordinator and worker nodes	Coordination overhead
Vector database	How are semantic matches retrieved?	Embedding index	Poor chunks or incompatible embeddings
Persistence	How does the system remember and resume?	Checkpointer and thread ID	Lost context or data leakage

---

Source Material

• [Week 5 live class slides](Live%20Class%20Slides_Multimodal%20Conversational%20System.pdf)

• [Week 5 roadmap](Roadmap_Week%205_SWEs.pdf)

• [AxiomCart project brief](Week%205_Project.txt)

• [AxiomCart reference implementation](axiomcart-ai-assistant/README.md)

• [Week 4 multi-agent lecture slides](../Week4_Multi_Agent_AI/Live%20Class%20Slides_Multi-Agent%20Systems_SWEs%20(1).pdf)